ASP.NET Core Authentication and Authorization

ASP.NET Core Identity

- ASP.NET Core Identity este un **sistem de membership care adauga functionalitate de autentificare** la o aplicatie ASP.NET Core
- **Utilizatorii pot sa isi creeze un cont cu informatiile stocate in Identity** sau **sa utilizeze un provider de autentificare extern** cum ar fi Facebook, Google, Microsoft sau Twitter
- Identity **poate sa fie configurat cu o baza de date SQL Server** pentru a stoca utilizatori, parole sau date de profil. Se pot utiliza si alte medii de stocare cum ar fi Azure Table Storage

Integrare ASP.NET Core Identity intr-o aplicatie existenta

- In cazul in care doriti sa pastrati tabelele Identity intr-o baza de date Sqlite instalati din NuGet **Microsoft.EntityFrameworkCore.Sqlite** version 10.0.3
- **Microsoft.AspNetCore.Identity.EntityFrameworkCore** version 10.0.3

1. Modificati clasa de context a aplicatiei pentru a mosteni din **IdentityDbContext<IdentityUser>**
2. Click dreapta pe numele proiectului in **Solution Explorer → Add → New Scaffolded Item**
3. In noua fereastra de dialog selectati **Identity → Add**
4. Bifati **Override all files**
5. Selectati din dropdown-ul **Data context class** contextul bazei de date
6. Apasati **Add**

Integrare ASP.NET Core Identity intr-o aplicatie existenta

- Stergeti intregistrarea anterioara in Program.cs

```
1   using AspNetCoreSecurityApp.Models;
2   using Microsoft.EntityFrameworkCore;
3   using Microsoft.AspNetCore.Identity;
4
5   var builder = WebApplication.CreateBuilder(args);
6
7   // Add services to the container.
8   builder.Services.AddControllersWithViews();
9   builder.Services.AddRazorPages();
10  builder.Services.AddDefaultIdentity<IdentityUser>(options => options.SignIn.RequireConfirmedAccount = true)
11      .AddRoles<IdentityRole>()
12      .AddEntityFrameworkStores<BloggingContext>();builder.Services.AddDbContext<BloggingContext>(options =>
13      options.UseSqlServer(builder.Configuration.GetConnectionString("BloggingDb")));
```

Integrare ASP.NET Core Identity intr-o aplicatie existenta

- Adaugati *Role* related services

```
var builder = WebApplication.CreateBuilder(args);

// Add services to the container.
builder.Services.AddControllersWithViews();
builder.Services.AddRazorPages();
builder.Services.AddDefaultIdentity<IdentityUser>(options => options.SignIn.RequireConfirmedAccount = true)
    .AddRoles<IdentityRole>()
    .AddEntityFrameworkStores<BloggingContext>();builder.Services.AddDbContext<BloggingContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("BloggingDb")));
```

Integrare ASP.NET Core Identity intr-o aplicatie existenta

- Adaugati *_LoginPartial* in **Layout page**

```
<body>
    <header>
        <nav class="navbar navbar-expand-sm navbar-toggleable-sm navbar-light bg-white border-bottom box-shadow mb-3">
            <div class="container-fluid">
                <a class="navbar-brand" asp-area="" asp-controller="Home" asp-action="Index">AspNetCoreSecurityApp</a>
                <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-target=".navbar-collapse" aria-contr
                        aria-expanded="false" aria-label="Toggle navigation">
                    <span class="navbar-toggler-icon"></span>
                </button>
                <div class="navbar-collapse collapse d-sm-inline-flex justify-content-between">
                    <ul class="navbar-nav flex-grow-1">
                        <li class="nav-item">
                            <a class="nav-link text-dark" asp-area="" asp-controller="Home" asp-action="Index">Home</a>
                        </li>
                        <li class="nav-item">
                            <a class="nav-link text-dark" asp-area="" asp-controller="Home" asp-action="Privacy">Privacy</a>
                        </li>
                    </ul>
                    <partial name="_LoginPartial" />
                </div>
            </div>
        </nav>
    </header>
    <div class="container">
        <main role="main" class="pb-3">
            @RenderBody()
        </main>
    </div>
```

ASP.NET Core Identity

```
// This method gets called by the runtime. Use this method to add services to the container.
0 references
public void ConfigureServices(IServiceCollection services)
{
    services.AddControllersWithViews();
    services.AddRazorPages();

    services.Add(new ServiceDescriptor(typeof(ILog), new MyConsoleLogger()));

    services.AddIdentity<IdentityUser, IdentityRole>()
        .AddEntityFrameworkStores<BloggingContext>();

    services.Configure<IdentityOptions>(options =>
    {
        // Password settings
        options.Password.RequireDigit = true;
        options.Password.RequiredLength = 8;
        options.Password.RequireNonAlphanumeric = false;
        options.Password.RequireUppercase = true;
        options.Password.RequireLowercase = false;
        options.Password.RequiredUniqueChars = 6;
```

```
    else
    {
        app.UseExceptionHandler("/Home/Error");
        // The default HSTS value is 30 days. You may want to change this for production scenarios,
        app.UseHsts();
    }
    app.UseHttpsRedirection();
    app.UseStaticFiles();

    app.UseRouting();

    app.UseAuthentication();
    app.UseAuthorization();

    app.UseEndpoints(endpoints =>
    {
        endpoints.MapControllerRoute(
            name: "default",
            pattern: "{controller=Home}/{action=Index}/{id?}");

        endpoints.MapRazorPages();
    });
}
```

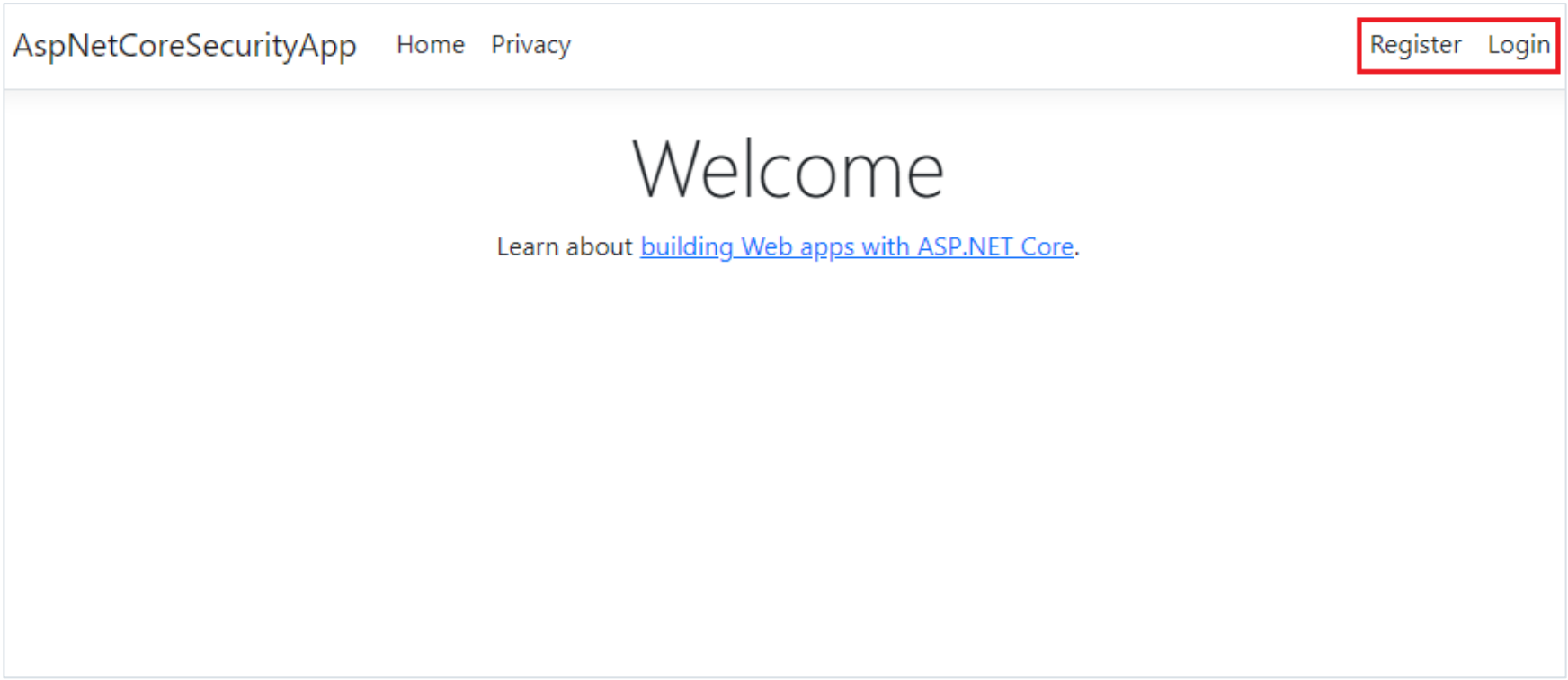
Integrare ASP.NET Core Identity intr-o aplicatie existenta

- Apelati ***UseAuthentication()*** dupa ***UseStaticFiles()*** pentru a inregistra pagina de autentificare
- Creati o noua migrare si apoi update database cu noile tabele *Identity*

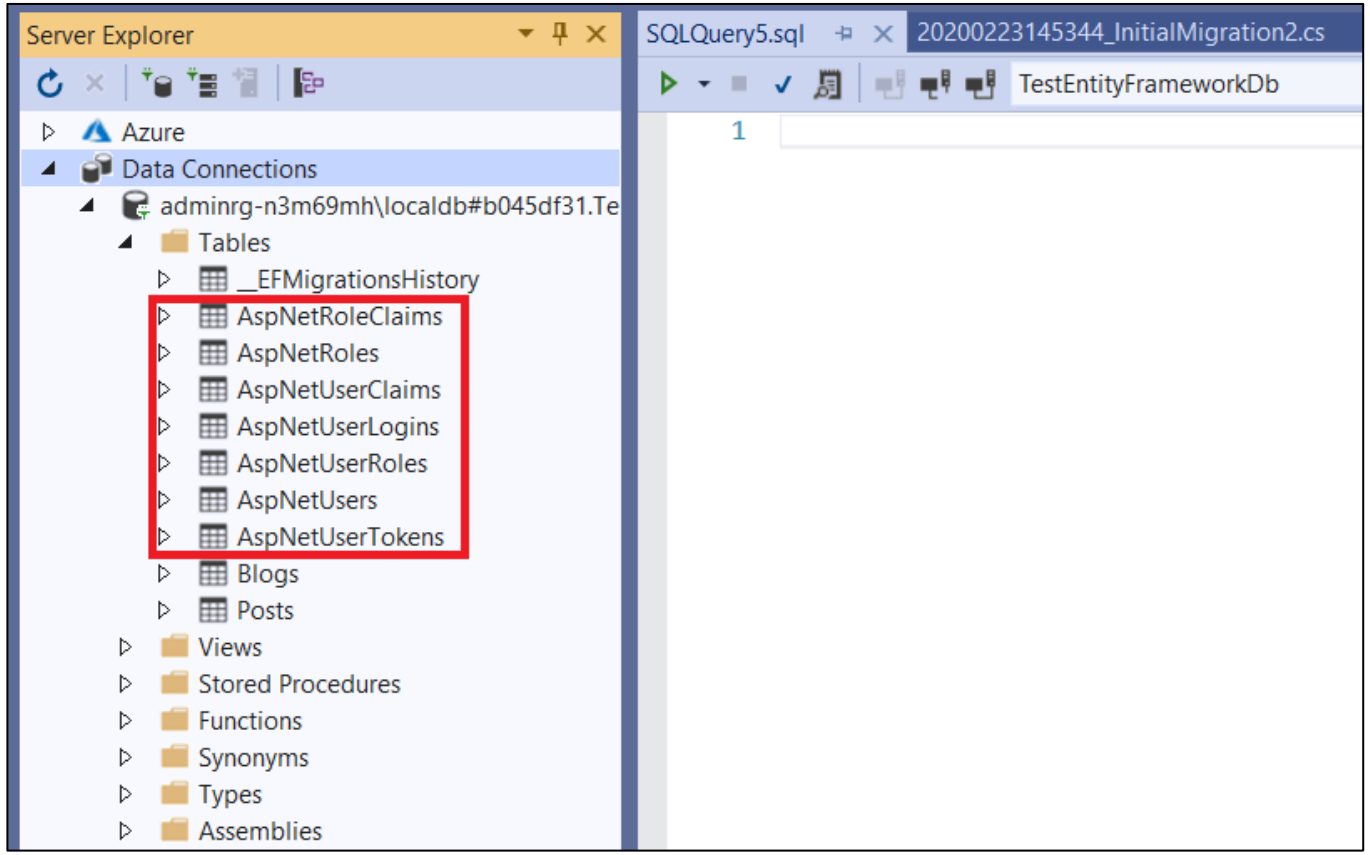
```
34  var app = builder.Build();
35
36  // Configure the HTTP request pipeline.
37  if (!app.Environment.IsDevelopment())
38  {
39      app.UseExceptionHandler("/Home/Error");
40      // The default HSTS value is 30 days. You may want to change this for production scenarios,
41      app.UseHsts();
42  }
43
44  app.UseHttpsRedirection();
45  app.UseStaticFiles();
46
47  app.UseRouting();
48  app.UseAuthentication();
49  app.UseAuthorization();
50
51  app.MapControllerRoute(
52      name: "default",
53      pattern: "{controller=Home}/{action=Index}/{id?}");
54
55  app.MapRazorPages();
56
57  app.Run();
```

Integrare ASP.NET Core Identity intr-o aplicatie existenta

- **Rulati** aplicatia.



ASP.NET Core Identity Tables



Tabele ASP.NET Core Identity

- **AspNetUsers** este utilizat pentru a stoca utilizatorii inregistrati ai aplicatiei
- **AspNetRoles** este utilizat pentru a defini rolurile
- **AspNetUserRoles** face legatura intre tabelele *AspNetUsers* si *AspNetRoles*. Este utilizat pentru a specifica rolurile pe care le au utilizatorii

Configurare ASP.NET Core Identity

- Se pot **configura constrangeri pentru parola** prin intermediul clasei *IdentityOptions*

```
public void ConfigureServices(IServiceCollection services)
{
    services.AddControllersWithViews();

    services.Add(new ServiceDescriptor(typeof(ILog), new MyConsoleLogger()));

    services.AddIdentity<IdentityUser, IdentityRole>()
        .AddEntityFrameworkStores<BloggingContext>();

    services.Configure<IdentityOptions>(options =>
    {
        // Password settings
        options.Password.RequireDigit = true;
        options.Password.RequiredLength = 8;
        options.Password.RequireNonAlphanumeric = false;
        options.Password.RequireUppercase = true;
        options.Password.RequireLowercase = false;
        options.Password.RequiredUniqueChars = 6;

        // Lockout settings
        options.Lockout.DefaultLockoutTimeSpan = TimeSpan.FromMinutes(30);
        options.Lockout.MaxFailedAccessAttempts = 10;
        options.Lockout.AllowedForNewUsers = true;

        // User settings
        options.User.RequireUniqueEmail = true;
    });
```

Configurare ASP.NET Core Identity

- Se pot **configura restrictii de acces** prin intermediul clasei *IdentityOptions*

```
public void ConfigureServices(IServiceCollection services)
{
    services.AddControllersWithViews();

    services.Add(new ServiceDescriptor(typeof(ILog), new MyConsoleLogger()));

    services.AddIdentity<IdentityUser, IdentityRole>()
        .AddEntityFrameworkStores<BloggingContext>();

    services.Configure<IdentityOptions>(options =>
    {
        // Password settings
        options.Password.RequireDigit = true;
        options.Password.RequiredLength = 8;
        options.Password.RequireNonAlphanumeric = false;
        options.Password.RequireUppercase = true;
        options.Password.RequireLowercase = false;
        options.Password.RequiredUniqueChars = 6;

        // Lockout settings
        options.Lockout.DefaultLockoutTimeSpan = TimeSpan.FromMinutes(30);
        options.Lockout.MaxFailedAccessAttempts = 10;
        options.Lockout.AllowedForNewUsers = true;

        // User settings
        options.User.RequireUniqueEmail = true;
    });
```

Configurare ASP.NET Core Identity

- Se pot **configura constrangeri legate de utilizatori** prin intermediul clasei *IdentityOptions*

```
public void ConfigureServices(IServiceCollection services)
{
    services.AddControllersWithViews();

    services.Add(new ServiceDescriptor(typeof(ILog), new MyConsoleLogger()));

    services.AddIdentity<IdentityUser, IdentityRole>()
        .AddEntityFrameworkStores<BloggingContext>();

    services.Configure<IdentityOptions>(options =>
    {
        // Password settings
        options.Password.RequireDigit = true;
        options.Password.RequiredLength = 8;
        options.Password.RequireNonAlphanumeric = false;
        options.Password.RequireUppercase = true;
        options.Password.RequireLowercase = false;
        options.Password.RequiredUniqueChars = 6;

        // Lockout settings
        options.Lockout.DefaultLockoutTimeSpan = TimeSpan.FromMinutes(30);
        options.Lockout.MaxFailedAccessAttempts = 10;
        options.Lockout.AllowedForNewUsers = true;

        // User settings
        options.User.RequireUniqueEmail = true;
    });
```

ASP.NET Core Identity Register

- In momentul in care se apasa *Register*, se apeleaza actiunea **RegisterModel.OnPostAsync.**
- Utilizatorul este creat de catre apelul *CreateAsync* asupra obiectului *_userManager.*
- *_userManager* este un serviciu responsabil cu gestiunea utilizatorilor si este injectat prin dependency injection in constructor

ASP.NET Core Identity Register

```
public async Task<IActionResult> OnPostAsync(string returnUrl = null)
{
    returnUrl = returnUrl ?? Url.Content("~/");
    ExternalLogins = (await _signInManager.GetExternalAuthenticationSchemesAsync()).ToList();
    if (ModelState.IsValid)
    {
        var user = new IdentityUser { UserName = Input.Email, Email = Input.Email };
        var result = await _userManager.CreateAsync(user, Input.Password);
        if (result.Succeeded)
        {
            _logger.LogInformation("User created a new account with password.");

            var code = await _userManager.GenerateEmailConfirmationTokenAsync(user);
            code = WebEncoders.Base64UrlEncode(Encoding.UTF8.GetBytes(code));
            var callbackUrl = Url.Page(
                "/Account/ConfirmEmail",
                pageHandler: null,
                values: new { area = "Identity", userId = user.Id, code = code },
                protocol: Request.Scheme);

            await _emailSender.SendEmailAsync(Input.Email, "Confirm your email",
                $"Please confirm your account by <a href='{HtmlEncoder.Default.Encode(callbackUrl)}
```

ASP.NET Core Identity Login

- Pagina de Login este afisata in momentul in care:
  - **se apasa butonul de Login din meniu**
  - **se incearca accesarea unei pagini pe care utilizatorul nu are permisiuni sa o vizualizeze**
  - **utilizatorul nu este logat si incearca sa acceseze o pagina care necesita ca utilizatorul sa fie logat**
- Cand se apasa butonul de *Log in* din pagina de Login se apeleaza actiunea **LoginModel.OnPostAsync**
- Utilizatorul este autentificat de catre apelul *PasswordSignInAsync* asupra obiectului *_signInManager*
- Clasa de baza Controller contine o proprietate **User** care ne permite sa o accesam din metodele din controllere. Obiectul User **expune diferite informatii despre utilizatorul logat**

ASP.NET Core Identity Login

```
public async Task<IActionResult> OnPostAsync(string returnUrl = null)
{
    returnUrl = returnUrl ?? Url.Content("~/");

    if (ModelState.IsValid)
    {
        // This doesn't count login failures towards account lockout
        // To enable password failures to trigger account lockout, set lockoutOnFailure: true
        var result = await _signInManager.PasswordSignInAsync(Input.Email, Input.Password, Input
        if (result.Succeeded)
        {
            _logger.LogInformation("User logged in.");
            return LocalRedirect(returnUrl);
        }
        if (result.RequiresTwoFactor)
        {
            return RedirectToPage("./LoginWith2fa", new { ReturnUrl = returnUrl, RememberMe = In
        }
        if (result.IsLockedOut)
        {
            _logger.LogWarning("User account locked out.");
            return RedirectToPage("./Lockout");
        }
```

ASP.NET Core Identity Logout

- Butonul de *Log out* apeleaza actiunea **LogoutModel.OnPost**

```
public async Task<IActionResult> OnPost(string returnUrl = null)
{
    await _signInManager.SignOutAsync();
    _logger.LogInformation("User logged out.");
    if (returnUrl != null)
    {
        return LocalRedirect(returnUrl);
    }
    else
    {
        return RedirectToPage();
    }
}
```

ASP.NET Core Identity Test

- **Paginile create de catre proiectul implicit pot sa fie accesate fara a fi necesara autentificarea**
- **Pentru a restrictiona accesul la o pagina** doar pentru utilizatorii autentificati trebuie specificat atributul **[Authorize]** In momentul in care se apasa pe link-ul de *Privacy* vom fi redirectionati catre pagina de Login

```
0 references
public HomeController(ILog log)
{
    _log = log;
}

0 references
public IActionResult Index()
{
    _log.info("Executing /home/index");

    return View();
}

[Authorize]
0 references
public IActionResult Privacy()
{
    return View();
}
```

ASP.NET Core Authorization

- Autorizarea in MVC **este controlata prin intermediul atributului AuthorizeAttribute** si parametrii sai
- **Aplicand atributul AuthorizeAttribute pe un controller sau actiune se limiteaza accesul la controller sau actiune doar pentru utilizatorii care sunt autentificati**
- In exemplul urmatorul, accesul la HomeController este permis doar utilizatorilor autentificati

ASP.NET Core Authorization

```
namespace AspNetCoreSecurityApp.Controllers
{
    [Authorize]
    3 references
    public class HomeController : Controller
    {
        private readonly ILogger<HomeController> _logger;

        0 references
        public HomeController(ILogger<HomeController> logger)
        {
            _logger = logger;
        }

        0 references
        public IActionResult Index()
        {
            return View();
        }

        0 references
        public IActionResult Privacy()
        {
            return View();
        }
```

ASP.NET Core Authorization

- Daca vrem sa aplicam **autorizarea la nivel de actiune**, trebuie sa **aplicam atributul AuthorizeAttribute pe actiunea respectiva**

```
public class HomeController : Controller
{
    private readonly ILogger<HomeController> _logger;

    0 references
    public HomeController(ILogger<HomeController> logger)
    {
        _logger = logger;
    }

    [Authorize]
    0 references
    public IActionResult Index()
    {
        return View();
    }

    0 references
    public IActionResult Privacy()
    {
        return View();
    }
```

ASP.NET Core Authorization

- In momentul de fata doar utilizatorii care sunt autentificati pot sa acceseze actiunea *About*
- Se poate utiliza **atributul AllowAnonymous pentru a permite accesul la o anumita actiune pentru utilizatorii care nu sunt autentificati**

ASP.NET Core Authorization

```
[Authorize]
3 references
public class HomeController : Controller
{
    private readonly ILogger<HomeController> _logger;

    0 references
    public HomeController(ILogger<HomeController> logger)
    {
        _logger = logger;
    }

    [AllowAnonymous]
    0 references
    public IActionResult Index()
    {
        return View();
    }

    0 references
    public IActionResult Privacy()
    {
        return View();
    }
```

ASP.NET Core Authorization

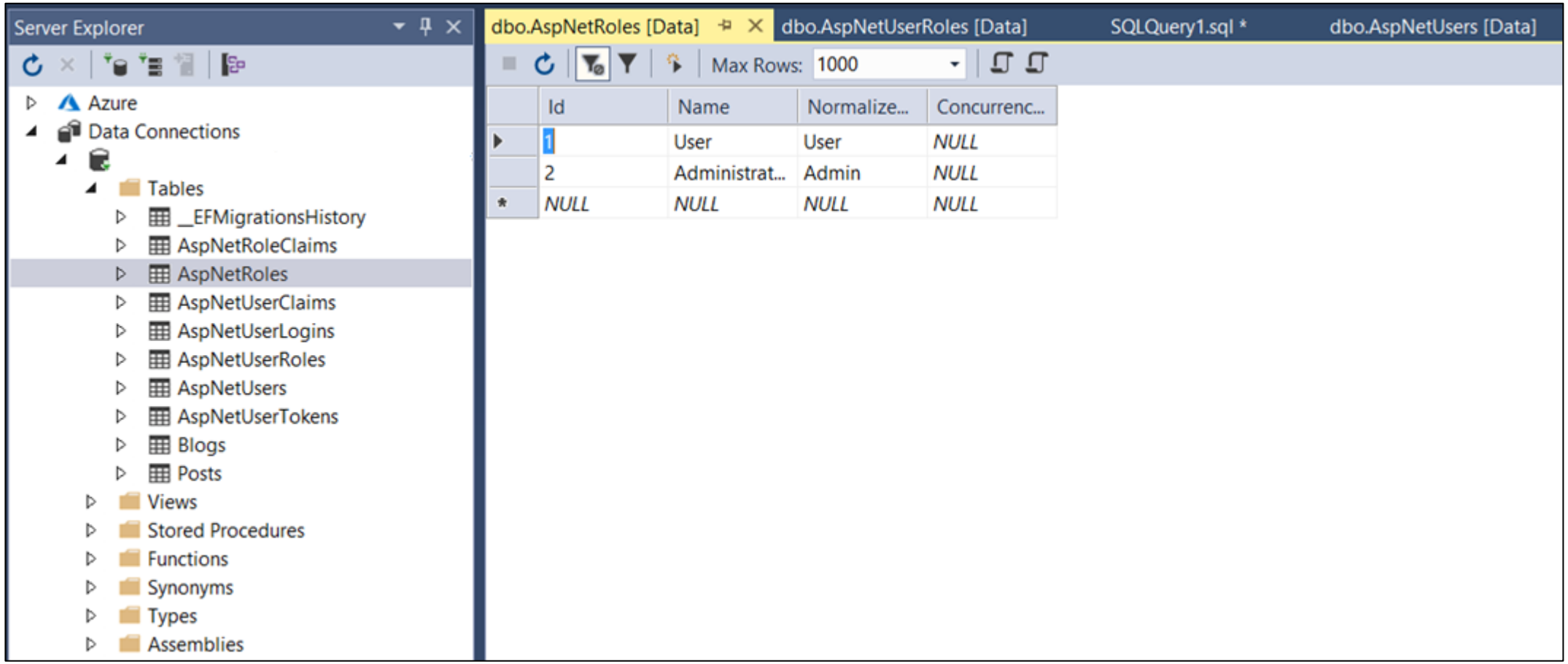
- Exemplul anterior va permite ca doar utilizatorii autentificati sa acceseze controllerul HomeController cu exceptia actiunii *Index* care este accesibila indiferent de statusul lor

ASP.NET Core Role-Based Authorization

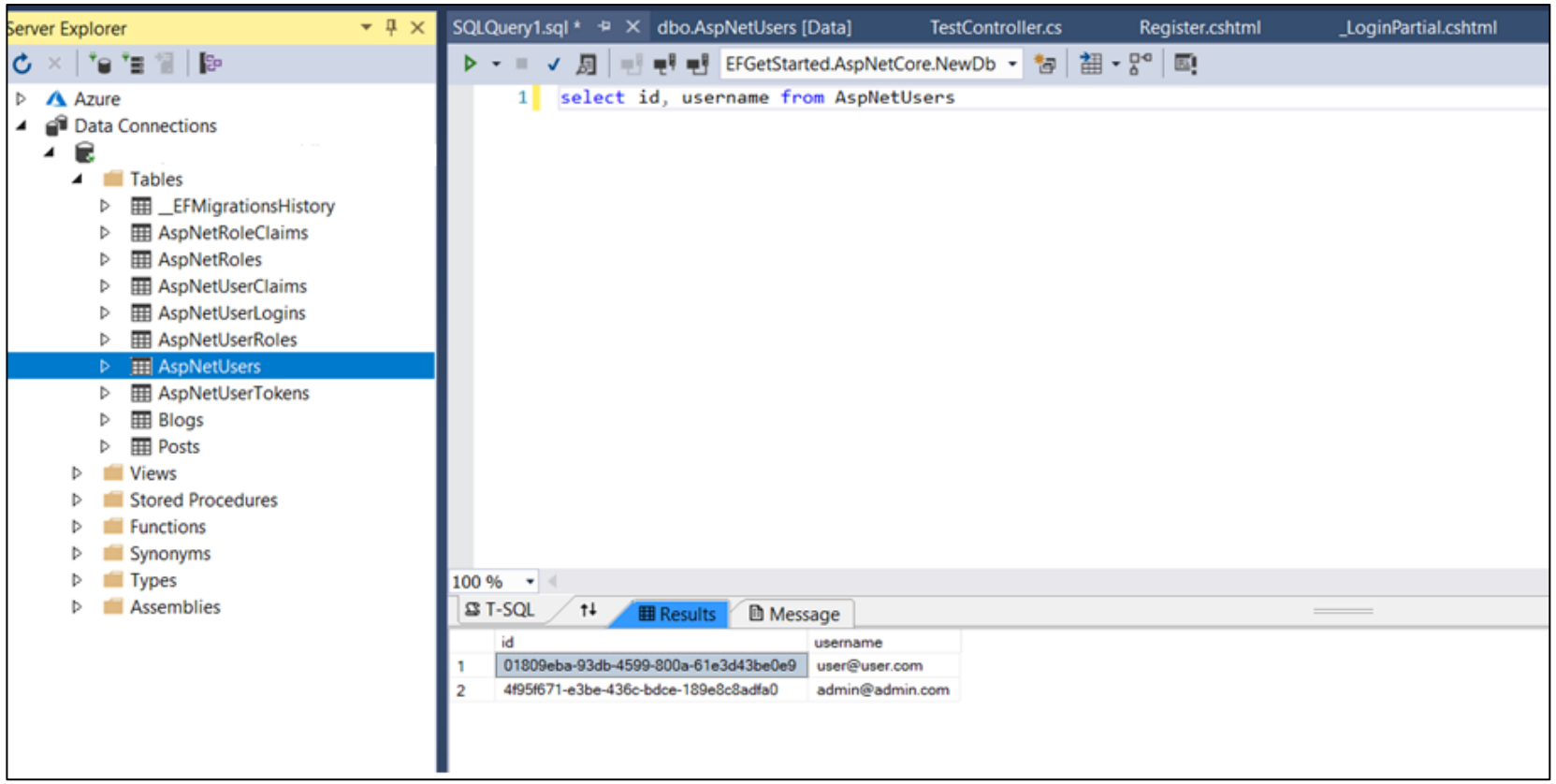
- **Cand o identitate (un utilizator) este creata poate sa apartina la unul sau mai multe roluri**. De exemplu, utilizatorul A poate sa aiba rolurile de *Administrator* si *User* in timp ce utilizatorul B poate sa aiba doar rolul *User*

- **Autorizarea bazata pe roluri este declarativa – programatorul seteaza acest lucru in cod pe un controller sau o actiune dintr-un controller, specificand rolurile pe care utilizatorul curent trebuie sa le detina pentru a accesa resursa ceruta**

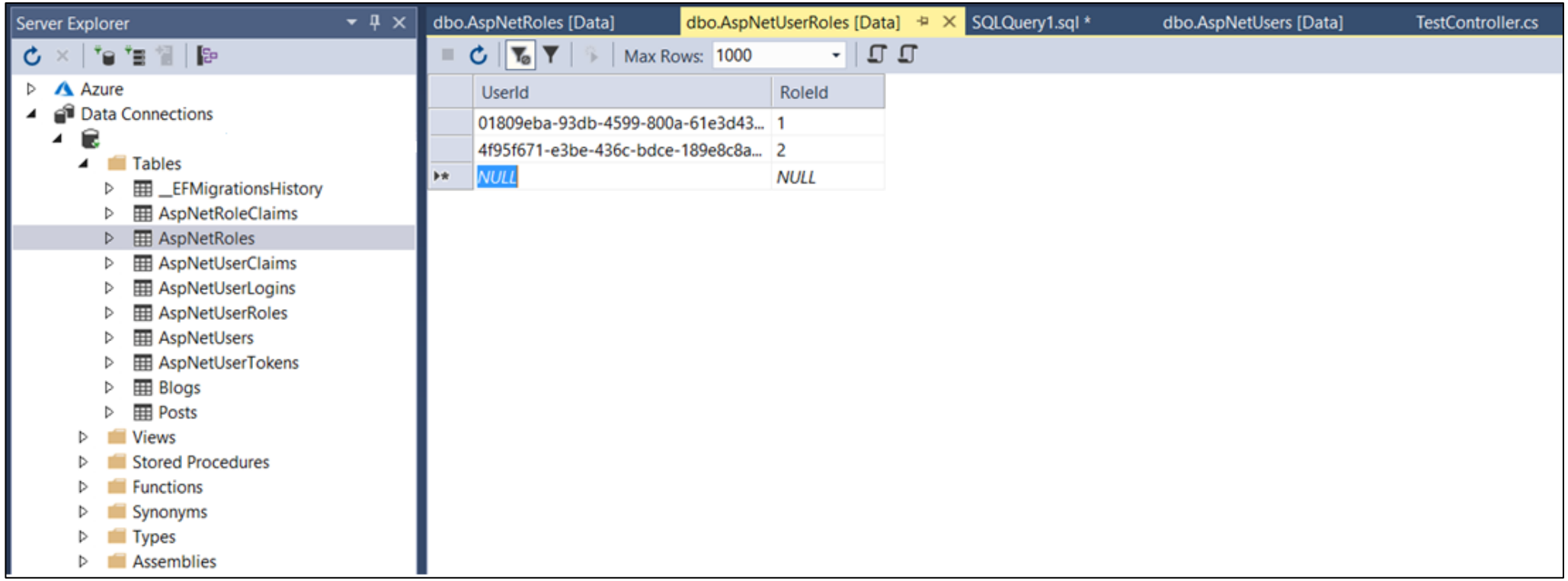
ASP.NET Core Role-Based Authorization – Definire Roluri



ASP.NET Core Role-Based Authorization – Utilizatori



ASP.NET Core Role-Based Authorization – Utilizatori Roluri



ASP.NET Core Role-Based Authorization

- In exemplul anterior **am introdus manual 2 roluri in tabelul *AspNetRoles*: User si Administrator**
- **Am specificat manual ce roluri are fiecare utilizator prin intermediul tabelei *AspNetUserRoles***
- Acest lucru se poate face automat din cod cu ajutorul clasei *RoleManager*

https://social.technet.microsoft.com/wiki/contents/articles/51333.asp-net-core-2-0-getting-started-with-identity-and-role-management.aspx

ASP.NET Core Role-Based Authorization

- Urmatorul cod **limiteaza accesul la actiunile din HomeController pentru utilizatorii care sunt membrii ai rolului Administrator**

```
[Authorize(Roles = "Administrator")]
3 references
public class HomeController : Controller
{
    private readonly ILogger<HomeController> _logger;

    0 references
    public HomeController(ILogger<HomeController> logger)
    {
        _logger = logger;
    }

    0 references
    public IActionResult Index()
    {
        return View();
    }

    0 references
    public IActionResult Privacy()
    {
        return View();
    }
```

ASP.NET Core Role-Based Authorization

- **Roluri multiple pot sa fie specificate prin virgula**

```
[Authorize(Roles = "Administrator,User")]
3 references
public class HomeController : Controller
{
    private readonly ILogger<HomeController> _logger;

    0 references
    public HomeController(ILogger<HomeController> logger)
    {
        _logger = logger;
    }

    0 references
    public IActionResult Index()
    {
        return View();
    }

    0 references
    public IActionResult Privacy()
    {
        return View();
    }
```

- **HomeController va putea fi accesat atat de catre administratori cat si de utilizatorii normali**

ASP.NET Core Role-Based Authorization

- **Daca sunt aplicate atribute multiple, atunci utilizatorul trebuie sa fie un membru al tuturor rolurilor specificate.** Urmatorul exemplu specifica faptul ca utilizatorul trebuie sa fie atat Administrator cat si User

```
[Authorize(Roles = "Administrator")]
[Authorize(Roles = "User")]
3 references
public class HomeController : Controller
{
    private readonly ILogger<HomeController> _logger;

    0 references
    public HomeController(ILogger<HomeController> logger)
    {
        _logger = logger;
    }

    0 references
    public IActionResult Index()
    {
        return View();
    }

    0 references
    public IActionResult Privacy()
    {
        return View();
    }
```

ASP.NET Core Role-Based Authorization

- Accesul se poate **limita la nivel de actiune**

```
[Authorize(Roles = "Administrator,User")]
3 references
public class HomeController : Controller
{
    private readonly ILogger<HomeController> _logger;

    0 references
    public HomeController(ILogger<HomeController> logger)
    {
        _logger = logger;
    }

    0 references
    public IActionResult Index()
    {
        return View();
    }

    [Authorize(Roles = "Administrator")]
    0 references
    public IActionResult Privacy()
    {
        return View();
    }
```

ASP.NET Core Role-Based Authorization

- In exemplul anterior atat utilizatorii cu rol de Administrator cat si cei cu rol de User pot sa acceseze HomeController si actiunile de *Index* si *Contact*, dar doar utilizatorii care au rol de Administrator pot sa acceseze actiunea *About*

Resurse

Scaffold Identity in ASP.NET Core projects | Microsoft Learn